Introduction to Graphs

Representations, Traversals, and Structural Patterns

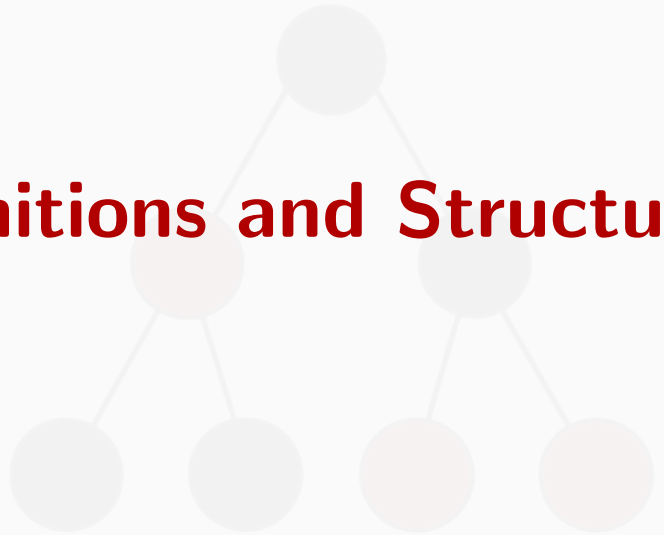
King's Competitive Programming Club

Workshop 4

Workshop Overview

1. Graph Definitions and Structural Families
2. Storing Graphs: Representations and Trade-Offs
3. Breadth-First Search (BFS) and Unweighted Shortest Paths
4. Depth-First Search (DFS) and Edge Classification
5. Try It Yourself
6. Conclusion

Graph Definitions and Structural Families



What Is a Graph?

A **graph** $G = (V, E)$ consists of:

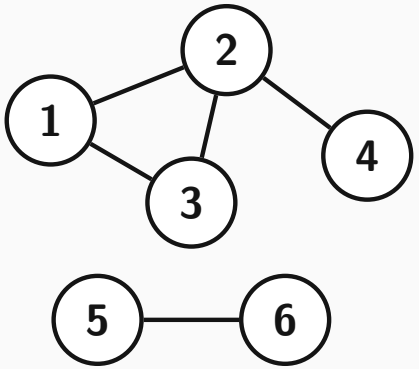
- → A set of **vertices** (nodes) $V = \{1, 2, \ldots, n\}$.
- → A set of **edges** E connecting pairs of vertices.

Example: $V = \{1, 2, 3, 4, 5, 6\}$,

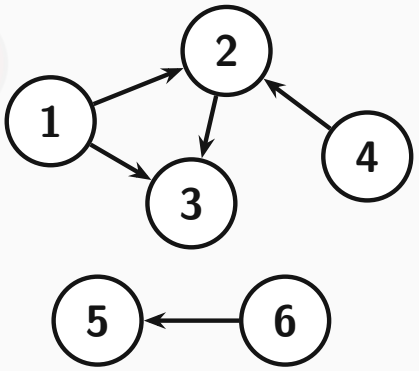
$E = \{(1, 2), (1, 3), (2, 3), (4, 2), (6, 5)\}$.

- → **Undirected:** edges are two-way $\{u, v\}$.
- → **Directed:** edges are one-way arrows $(u \to v)$.

Undirected graph



Directed graph



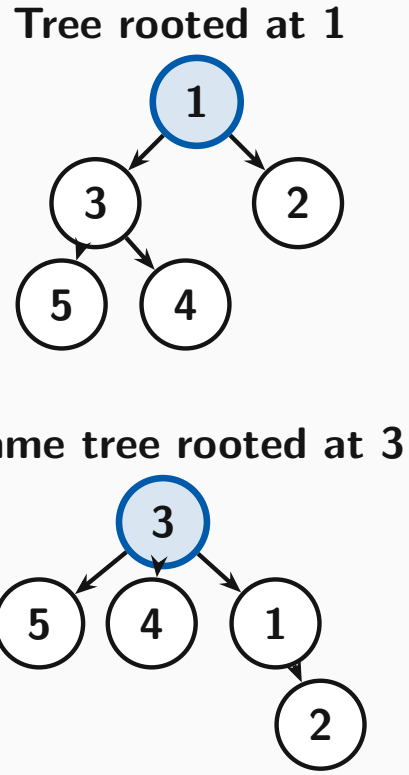
Paths, Cycles, and Degrees

- **Path:** A sequence of vertices $(v_1, v_2, \ldots, v_k)$ with an edge between each consecutive pair (v_i, v_{i+1}). A path is **simple** if all vertices are distinct.
- **Cycle:** A path of length ≥ 3 where $v_1 = v_k$ and $v_1, \ldots, v_{k-1}$ are distinct.
- **Degree** $\deg(v)$**:** The number of edges incident to vertex v.
  - In directed graphs, we split into **in-degree** $\deg^-(v)$ and **out-degree** $\deg^+(v)$.
- **Handshaking Lemma:** Every undirected edge contributes 1 to the degree of both endpoints:

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Connectivity, Trees, and Rooted Trees

- → **Connected Graph:** A path exists between any two vertices $u, v \in V$.
  - → Maximal connected subgraphs are called **connected components**.
- → **Tree:** A connected, acyclic undirected graph.
  - → Equivalent: connected with $|E| = |V| - 1$.
  - → Equivalent: **unique** simple path between any two vertices.
- → **Rooted Tree:** Fixing one special **root** node r orients all edges as parent → child.





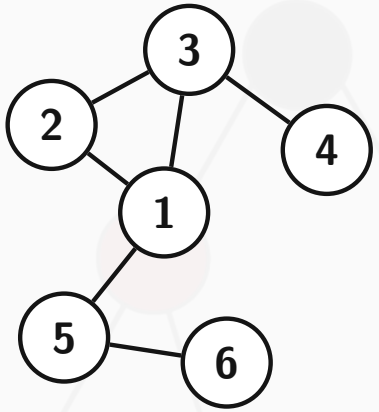
Storing Graphs: Representations and Trade-Offs

Three Ways to Represent a Graph

1. Edge List

Store E as pairs (u, v):

```
2 3
1 2
3 1
3 4
1 5
6 5
```



2. Adjacency List

For each u, list neighbours:

```
1:  2, 3, 5
2:  3, 1
3:  2, 1, 4
4:  3
5:  1, 6
6:  5
```

3. Adjacency Matrix: An $N \times N$ table A where $A[u][v] = 1$ if $(u, v) \in E$, else 0.

Adjacency List in Code (C++ and Python)

C++ (vector<int>)

```cpp
const int MAXN = 100005;
vector<int> g[MAXN];

int main() {
  int n, m;
  cin >> n >> m;
  for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u); // 2-way
  }
}
```

Python (list of lists)

```python
import sys
input = sys.stdin.readline

n, m = map(int, input().split())
g = [[] for _ in range(n + 1)]
for _ in range(m):
  u, v = map(int, input().split())
  g[u].append(v)
  g[v].append(u) # undirected
```

Total integers stored across all lists: $2|E|$ (or $|E|$ if directed).

Adjacency Matrix in Code

```
const int MAXN = 1005;
bool g[MAXN][MAXN];

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        g[u][v] = true;
        g[v][u] = true; // undirected
    }
}
```

Matrix $g[u][v]$ for the 6-node graph

	1	2	3	4	5	6
1	0	1	1	0	1	0
2	1	0	1	0	0	0
3	1	1	0	1	0	0
4	0	0	1	0	0	0
5	1	0	0	0	0	1
6	0	0	0	0	1	0

Symmetric ($g[u][v] = g[v][u]$) when undirected.

Comparison: Memory and Operations

Memory ($N = |V|$, $E = |E|$):

Structure	N=1k, E=1k	N=1k, E=100k	N=5k, E=200k	N=100k, E=200k
Adj. List $O(N+E)$	$\approx$0.01 MB	$\approx$1 MB	$\approx$2 MB	$\approx$2 MB
Adj. Matrix $O(N^2)$	$\approx$1 MB	$\approx$1 MB	$\approx$25 MB	$\approx$10 **GB**

Operation Time Complexities:

Operation	Adjacency List	Adjacency Matrix	Edge List
Does edge (u, v) exist?	$O(\deg(u))$	$O(1)$	$O(E)$
Iterate neighbours of u	$O(\deg(u))$	$O(N)$	$O(E)$
Inspect all edges	$O(N + E)$	$O(N^2)$	$O(E)$
Add / delete edge (u, v)	$O(1)$ / $O(\deg(u))$	$O(1)$ / $O(1)$	$O(1)$ / $O(E)$
Add / delete vertex u	$O(1)$ / $O(N + E)$	$O(N)$ / $O(N)$	$O(1)$ / $O(E)$

Try It Yourself: Representations and Degrees

Suppose an undirected simple graph has $N = 100\,000$ vertices and $M = 150\,000$ edges.

1. What is the sum of $\deg(v)$ over all 100 000 vertices?
2. If you store the graph in an adjacency list `g[1..N]`, how many total loop iterations does

   $$\texttt{for } u = 1 \ldots N: \quad \texttt{for } v \text{ in } g[u]: \ldots$$

   perform, and why is this much faster than scanning an adjacency matrix?
3. Can an undirected simple graph on 5 vertices have vertex degrees $3, 3, 3, 2, 2$?

Discuss with a neighbour.

Try It Yourself: Representations and Degrees (Answers)

1. **Sum of degrees:** By the Handshaking Lemma,

$$\sum_{v=1}^{100\,000} \deg(v) = 2M = 2 \times 150\,000 = 300\,000.$$

2. **Adjacency list loop iterations:** The inner loop runs $\deg(u)$ times for vertex u, so total iterations equal $\sum_u \deg(u) = 300\,000$ (3×10^5 operations), compared to $N^2 = 10^{10}$ operations in an adjacency matrix.
3. **Degree sequence** $3, 3, 3, 2, 2$**:** Impossible! Their sum is $3 + 3 + 3 + 2 + 2 = 13$, which is odd, violating $\sum \deg(v) = 2|E|$.

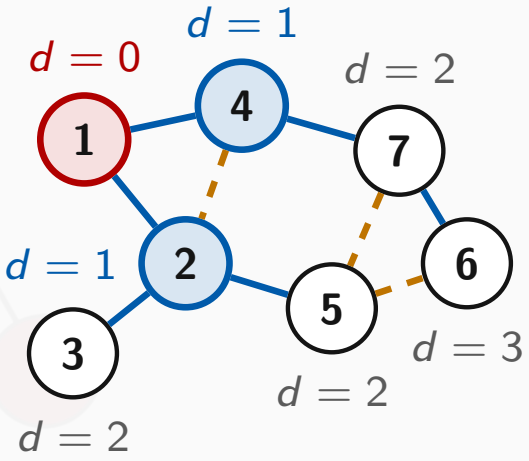


Breadth-First Search and Unweighted Shortest Paths

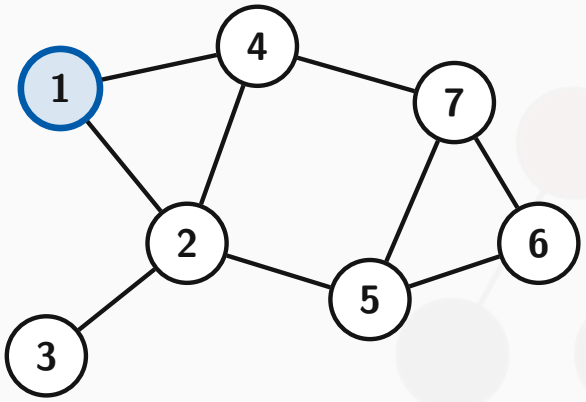
Breadth-First Search (BFS): Layer-by-Layer Exploration

Core Idea:

- → Start at source vertex s (layer 0).
- → Visit all neighbours at distance 1, then distance 2, then distance 3, and so on.
- → Maintain a **FIFO queue** q of discovered vertices whose neighbours still need to be inspected.
- → Mark a vertex `visited` **as soon as it is pushed** into q so it is never enqueued twice.



BFS Step-by-Step Walkthrough



Initialise

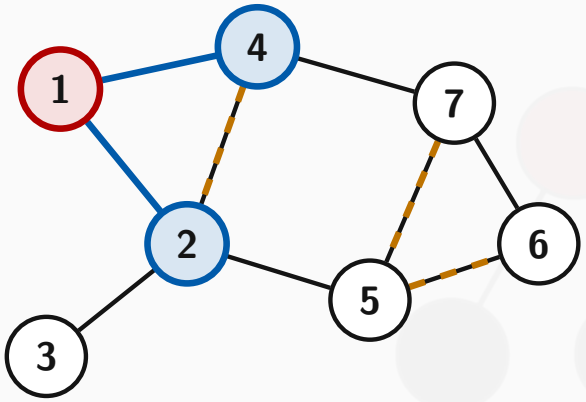
Push source 1 and mark it visited.

Queue after this step:

[1]

Discovered: 1

BFS Step-by-Step Walkthrough



Pop 1

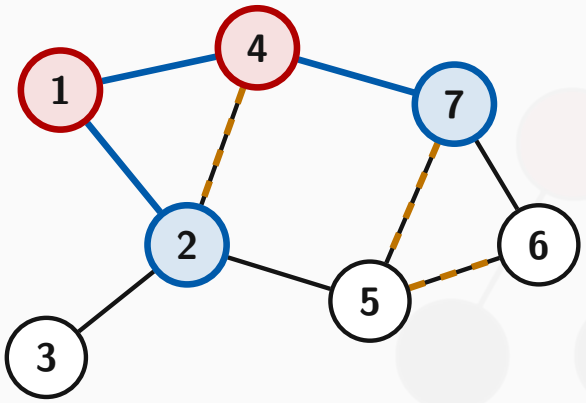
Discover 4 and 2; both have distance 1.

Queue after this step:

$[4, 2]$

Discovered: $1, 4, 2$

BFS Step-by-Step Walkthrough



Pop 4

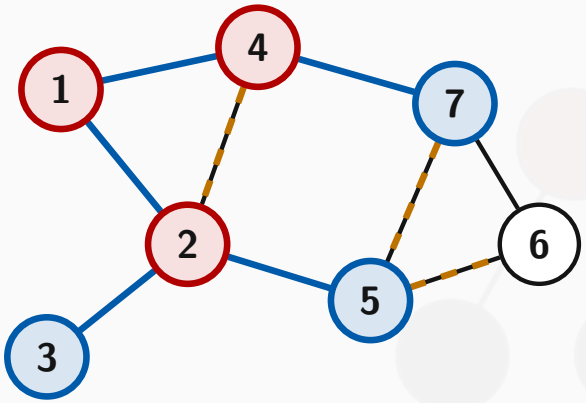
1 and 2 are visited; discover 7 at distance 2.

Queue after this step:

$[2, 7]$

Discovered: $1, 4, 2, 7$

BFS Step-by-Step Walkthrough



Pop 2

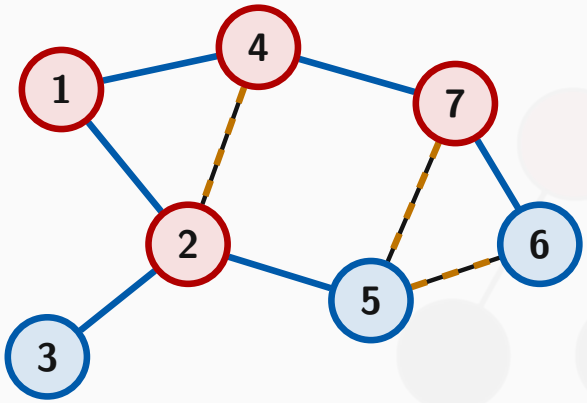
Discover 3 and 5 at distance 2.

Queue after this step:

$[7, 3, 5]$

Discovered: $1, 4, 2, 7, 3, 5$

BFS Step-by-Step Walkthrough



Pop 7

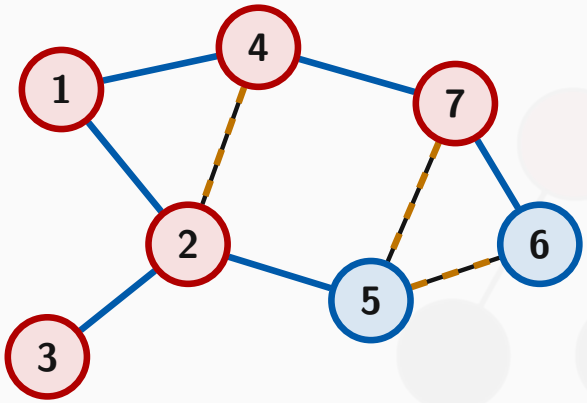
Discover 6 at distance 3.

Queue after this step:

$[3, 5, 6]$

Discovered: $1, 4, 2, 7, 3, 5, 6$

BFS Step-by-Step Walkthrough



Pop 3

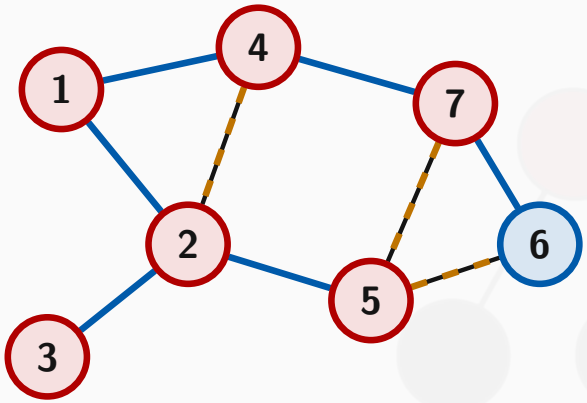
All its neighbours are already visited.

Queue after this step:

$[5, 6]$

Discovered: $1, 4, 2, 7, 3, 5, 6$

BFS Step-by-Step Walkthrough



Pop 5

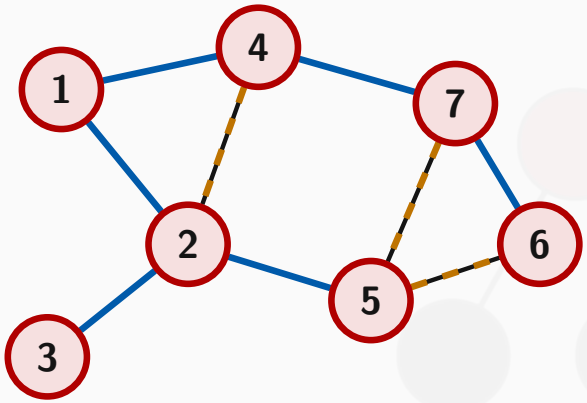
6 is already discovered, so $(5, 6)$ is not a tree edge.

Queue after this step:

[6]

Discovered: $1, 4, 2, 7, 3, 5, 6$

BFS Step-by-Step Walkthrough



Pop 6

The queue is empty; BFS is complete.

Queue after this step:

[]

BFS order: 1, 4, 2, 7, 3, 5, 6

BFS with Shortest Paths and Parent Reconstruction

C++ Implementation

```
vector<int> dist(n + 1, -1);
vector<int> parent(n + 1, 0);
queue<int> q;
q.push(start);
dist[start] = 0;

while (!q.empty()) {
  int v = q.front();
  q.pop();
  for (int u : g[v]) {
    if (dist[u] == -1) {
      dist[u] = dist[v] + 1;
      parent[u] = v;
      q.push(u);
    }
  }
}
```

Properties of BFS:

- → Tree edges form a **shortest-path tree** rooted at `start`.
- → Every vertex is enqueued at most once and each edge is inspected twice: $\mathbf{O(N + E)}$ time and space.
- → **Path reconstruction** (*Message Route*): follow `cur = parent[cur]` from target t back to `start`, then reverse.

Try It Yourself: BFS Shortest Path

Consider the 7-vertex graph from the BFS walkthrough (edges: $(1,4),(1,2),(4,2),(4,7),(2,3),(2,5),(7,5),(7,6),(5,6)$).

1. What is the shortest distance `dist[6]` from vertex 1 to vertex 6, and what is the parent chain `6 -> parent[6] -> ... -> 1`?
2. Why is the edge $(5,6)$ a **cross edge** rather than a tree edge when 7 is popped before 5?
3. What bug occurs if you set `visited[u] = true` when *popping* u from the queue instead of when *pushing* u?

Try It Yourself: BFS Shortest Path (Answers)

1. **Shortest path to** 6**:** `dist`$[6] = 3$. Following parents gives $6 \leftarrow 7 \leftarrow 4 \leftarrow 1$, so the shortest path is $1 \rightarrow 4 \rightarrow 7 \rightarrow 6$.
2. **Why** $(5, 6)$ **is a cross edge:** Both 7 and 5 are in layer $d = 2$, with 7 enqueued before 5. When 7 is popped, it discovers 6 ($d[6] = 3$). Later, when 5 is popped, 6 is already visited, so $(5, 6)$ is a non-tree cross edge.
3. **Marking visited on pop instead of push:** The same vertex can be pushed into the queue multiple times by different neighbours in the previous layer, causing duplicate work and up to $O(E)$ queue blowup (and incorrect parent overwrites).

Depth-First Search and Edge Classification



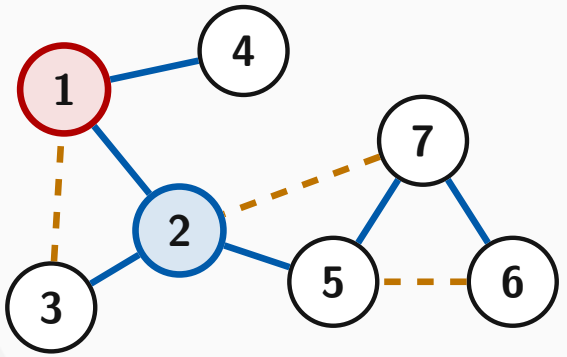
Depth-First Search (DFS): Plunge and Backtrack

Core Idea:

→ Follow one branch as deep as possible until stuck, then **backtrack**.

→ Naturally expressed via recursion (or an explicit LIFO stack).

```
void dfs(int v) {
    vis[v] = true;
    tin[v] = ++timer;
    for (int u : g[v]) {
        if (!vis[u]) {
            dfs(u);
        }
    }
    tout[v] = ++timer;
}
```



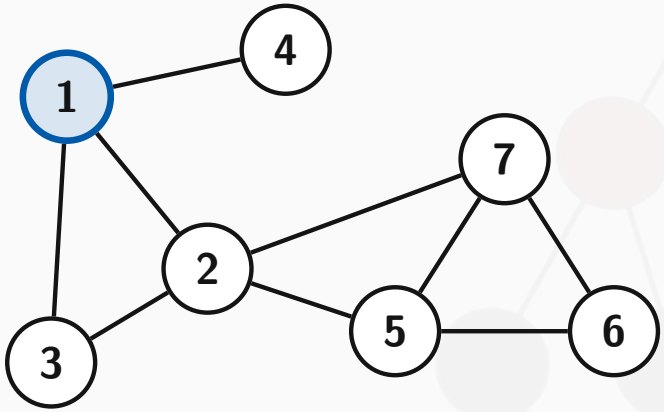
DFS Entry (`tin`) order:

1, 4, 2, 3, 5, 7, 6

DFS Exit (`tout`) order:

4, 3, 6, 7, 5, 2, 1

DFS Step-by-Step: Follow a Branch, Then Backtrack



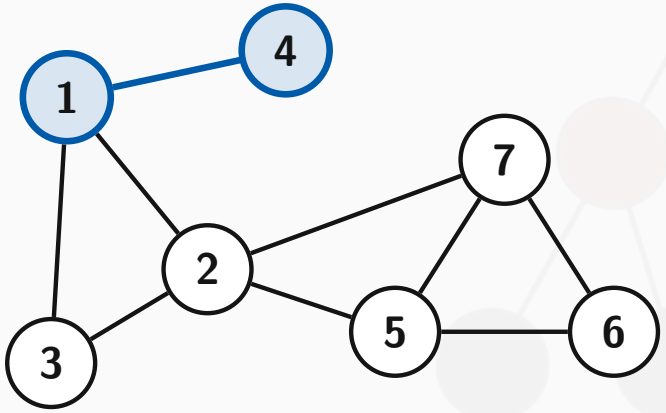
Call dfs(1)

Start at vertex 1.

Call stack:

[1]

DFS Step-by-Step: Follow a Branch, Then Backtrack



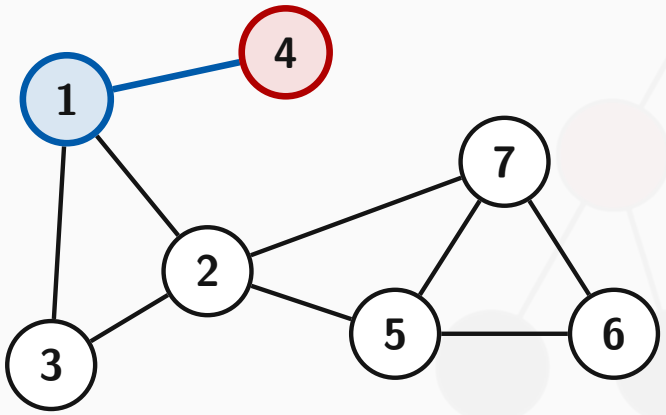
Call dfs(4)

Follow the first neighbour of 1.

Call stack:

$[1, 4]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



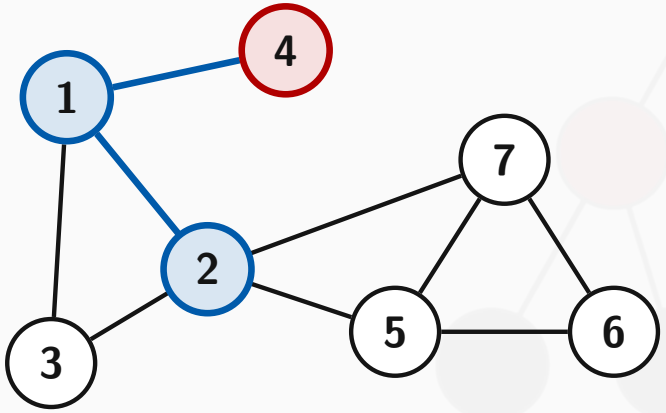
Return from 4

Its only neighbour, 1, is its parent.

Call stack:

[1]

DFS Step-by-Step: Follow a Branch, Then Backtrack



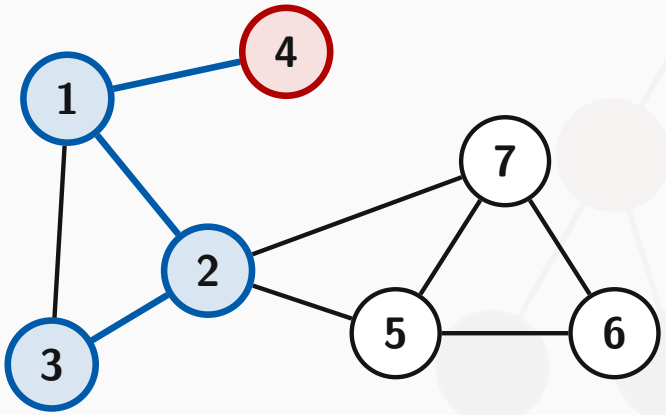
Call dfs(2)

Back at 1, take the next unvisited neighbour.

Call stack:

$[1, 2]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



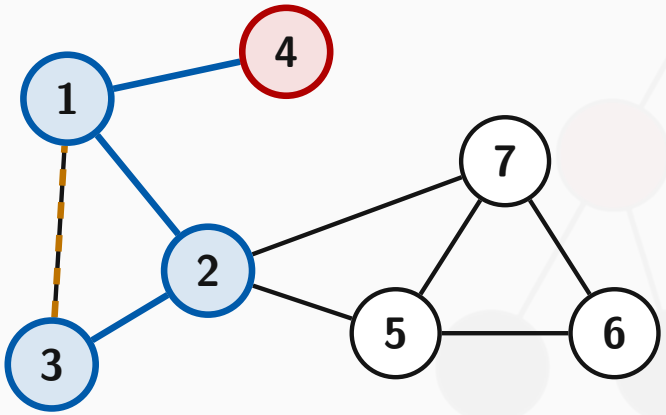
Call dfs(3)

From 2, follow the next unvisited neighbour.

Call stack:

$[1, 2, 3]$

DFS Step-by-Step: Follow a Branch, Then Backtrack

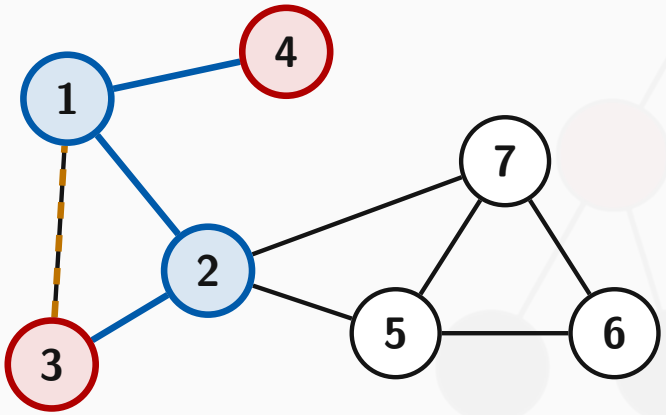


Back edge $(3, 1)$
1 is already on the recursion stack.

Call stack:

$[1, 2, 3]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



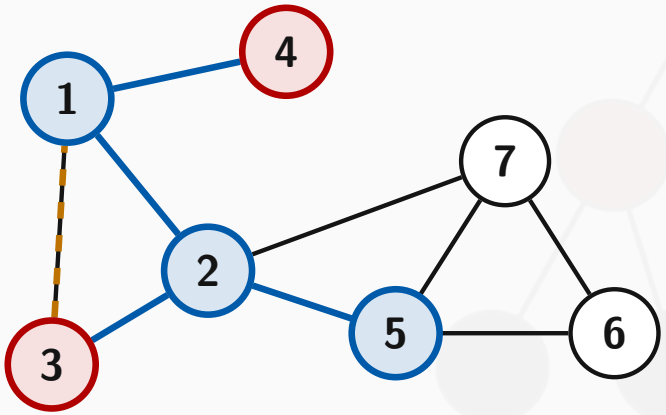
Return from 3

Finish this branch and resume at 2.

Call stack:

$[1, 2]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



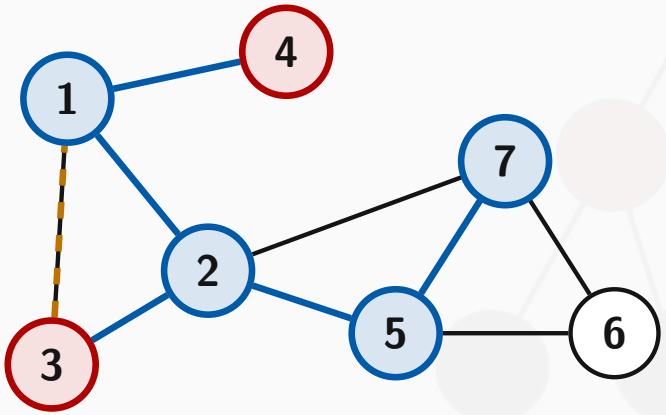
Call dfs(5)

Continue from 2 to its next unvisited neighbour.

Call stack:

$[1, 2, 5]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



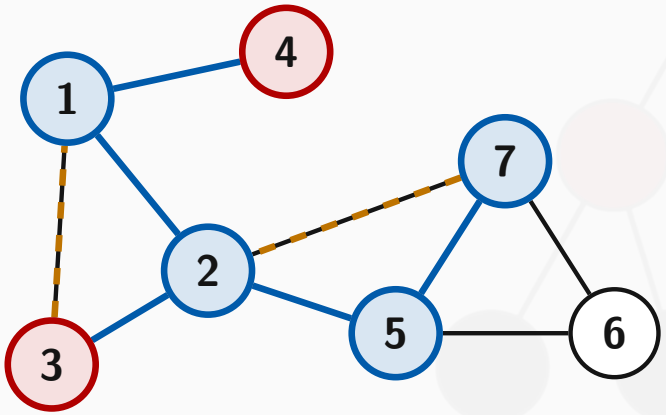
Call dfs(7)

Follow the branch through 5.

Call stack:

$[1, 2, 5, 7]$

DFS Step-by-Step: Follow a Branch, Then Backtrack

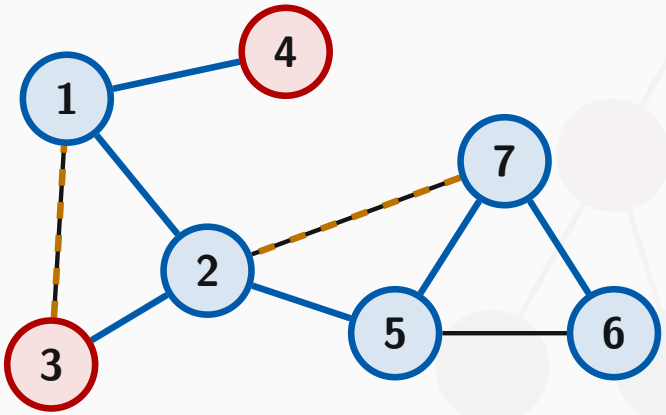


Back edge $(7, 2)$
2 is an ancestor already on the stack.

Call stack:

$[1, 2, 5, 7]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



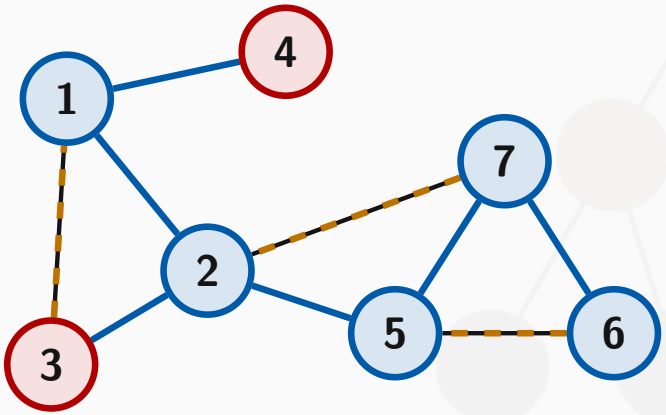
Call dfs(6)

From 7, follow its unvisited neighbour.

Call stack:

$[1, 2, 5, 7, 6]$

DFS Step-by-Step: Follow a Branch, Then Backtrack

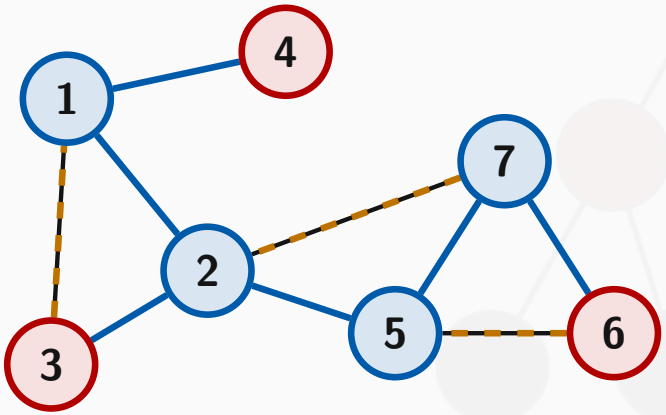


Back edge $(6, 5)$
5 is already on the recursion stack.

Call stack:

$[1, 2, 5, 7, 6]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



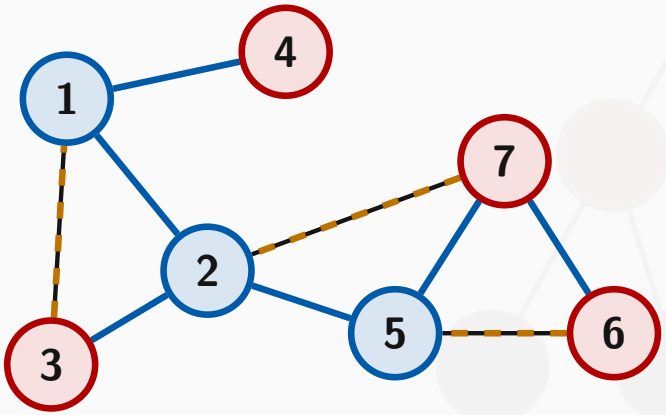
Return from 6

No unvisited neighbours remain.

Call stack:

$[1, 2, 5, 7]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



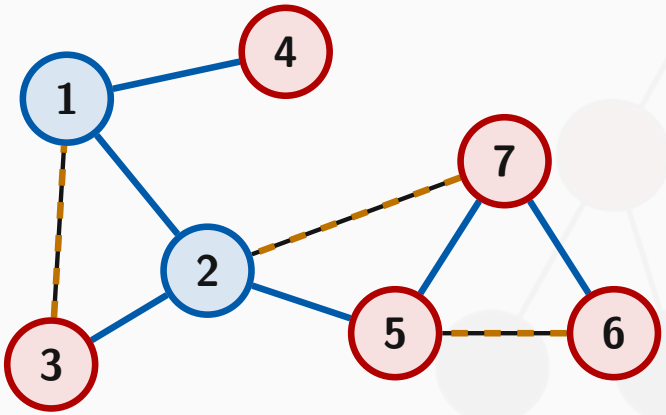
Return from 7

Resume at 5.

Call stack:

$[1, 2, 5]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



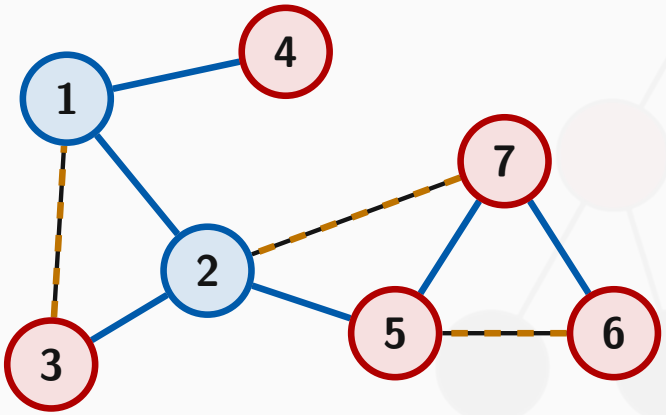
Return from 5

Resume at 2.

Call stack:

$[1, 2]$

DFS Step-by-Step: Follow a Branch, Then Backtrack



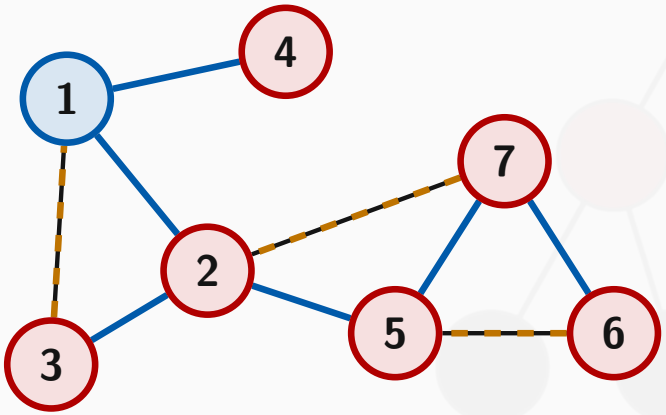
Return from 2

Resume at 1.

Call stack:

[1]

DFS Step-by-Step: Follow a Branch, Then Backtrack



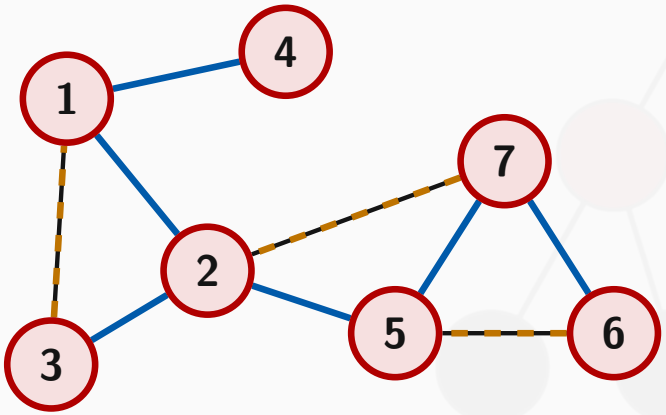
Neighbour 3 already visited

Skip it and finish 1.

Call stack:

[1]

DFS Step-by-Step: Follow a Branch, Then Backtrack



DFS complete

Every reachable vertex has been visited.

Call stack:

[]

Entry order: $1, 4, 2, 3, 5, 7, 6$
Exit order: $4, 3, 6, 7, 5, 2, 1$

DFS Properties and Edge Classification

- → **Subtree Property:** Once DFS exits vertex v, it has visited *every* vertex reachable from v along unvisited paths. Vertex u is in v's DFS subtree iff $\text{tin}[v] \leq \text{tin}[u] \leq \text{tout}[u] \leq \text{tout}[v]$.
- → **Edge Types in Undirected Graphs:** Every edge is either:
  1. a **Tree Edge** (discovers a new vertex u from v), or
  2. a **Back Edge** (connects v to an already-visited ancestor $u \neq \text{parent}[v]$).
- → **Edge Types in Directed Graphs:** In addition to tree and back edges, directed graphs can have **forward edges** (to a proper descendant) and **cross edges** (to a previously finished branch).
- → **Complexity:** Visits every vertex and edge once: $\mathbf{O(N + E)}$ time and $\mathbf{O(N)}$ recursion stack.

Try It Yourself: DFS Trace and Back-Edges

In the 7-vertex DFS walkthrough graph, the adjacency lists are:

$$1 : [4, 2, 3], \quad 2 : [1, 3, 5, 7], \quad 3 : [2, 1], \quad 4 : [1],$$
$$5 : [2, 7, 6], \quad 6 : [7, 5], \quad 7 : [2, 5, 6].$$

1. When DFS is at vertex 3 (called from 2, which was called from 1), it inspects neighbour 1. Why does $(3, 1)$ prove the existence of a simple cycle?
2. List all three back-edges encountered during `dfs`(1) and the simple cycle each back-edge closes.

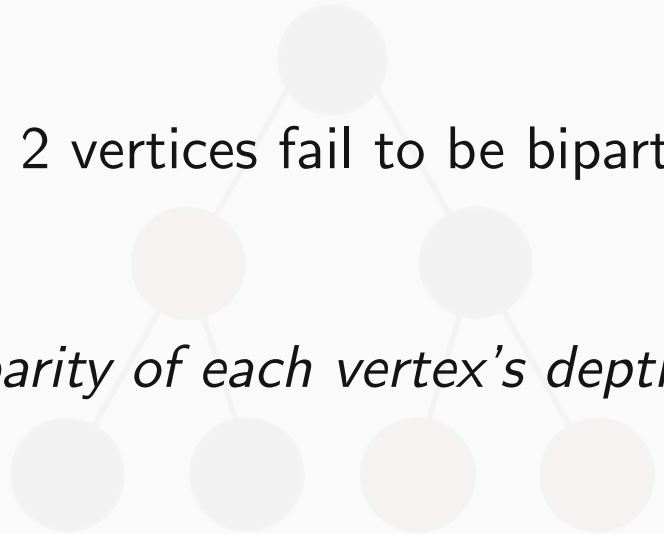
Try It Yourself: DFS Trace and Back-Edges (Answers)

1. **Why $(3, 1)$ proves a cycle:** At vertex 3, $\texttt{parent}[3] = 2$. Neighbour 1 is already visited and $1 \neq \texttt{parent}[3]$, so 1 is an ancestor of 3 in the DFS tree. Combining the tree path $1 \to 2 \to 3$ with the back-edge $(3, 1)$ gives the simple cycle $1 - 2 - 3 - 1$.
2. **All three back-edges:**
   - $\to$ $(3, 1)$ closes cycle $1 - 2 - 3 - 1$.
   - $\to$ $(7, 2)$ closes cycle $2 - 5 - 7 - 2$.
   - $\to$ $(6, 5)$ closes cycle $5 - 7 - 6 - 5$.



Try It Yourself

Try It Yourself: Tree Bipartiteness



Can a tree with $N \geq 2$ vertices fail to be bipartite? Explain why or why not.

Hint: Consider the parity of each vertex's depth from a chosen root.

Try It Yourself: Tree Bipartiteness (Answer)

No. Root the tree anywhere and colour each vertex by the parity of its depth.

Every tree edge joins a parent to a child, whose depths differ by exactly one, so its endpoints have opposite colours. Thus this is a valid 2-colouring. Equivalently, a tree has no cycles, and hence no odd cycles.

Try It Yourself: Counting Paths in a DAG

Suppose a DAG has vertices $\{1, 2, 3, 4\}$ and directed edges

$$1 \to 2, \quad 1 \to 3, \quad 2 \to 3, \quad 2 \to 4, \quad 3 \to 4.$$

Find a valid topological ordering and calculate the number of directed paths from vertex 1 to vertex 4.

Try It Yourself: Counting DAG Paths (Answer)

A valid topological ordering is $[1, 2, 3, 4]$. Initialise $\mathrm{dp}[1] = 1$ and process the edges in that order:

$$\begin{aligned} \mathrm{dp}[2] &= 1, \\ \mathrm{dp}[3] &= \mathrm{dp}[1] + \mathrm{dp}[2] = 2, \\ \mathrm{dp}[4] &= \mathrm{dp}[2] + \mathrm{dp}[3] = 1 + 2 = \mathbf{3}. \end{aligned}$$

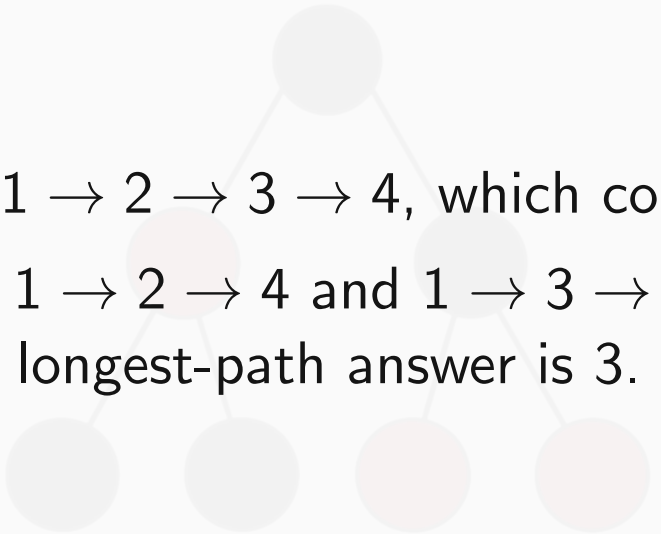
The three routes are $1 \to 2 \to 4$, $1 \to 3 \to 4$, and $1 \to 2 \to 3 \to 4$.

Try It Yourself: Longest Path in a DAG

Using the same DAG, find the length (number of edges) of the longest directed path from vertex 1 to vertex 4.

Hint: Compare the direct routes with the route that passes through both vertices 2 and 3.

Try It Yourself: Longest DAG Path (Answer)



The longest route is $1 \to 2 \to 3 \to 4$, which contains **3 edges**.

The other routes are $1 \to 2 \to 4$ and $1 \to 3 \to 4$, each with 2 edges. Therefore the longest-path answer is 3.



Conclusion

Summary: Matching Problems to Graph Techniques

Goal	Key Invariant	Method
Connect components	Reachability classes	DFS/BFS, $O(N+E)$
Unweighted shortest path	BFS layers	BFS + `parent[]`, $O(N+E)$
Find undirected cycle	Non-parent back-edge	DFS + `parent[]`, $O(N+E)$
Split into two teams	No odd cycle	2-colouring, $O(N+E)$
Component is a cycle	Connected; degree 2	Degree check, $O(N+E)$
Paths / longest in DAG	Topological order	Kahn / DFS DP, $O(N+E)$
Tree diameter	Farthest node is an endpoint	Two-BFS/DFS, $O(N)$
Same-colour line rides	Auxiliary line nodes	Auxiliary BFS, $O(N+E)$